TECMILENIO

# Realizar la preparación de los datos, el entrenamiento de un modelo y el registro de los resultados.

**Esta es la ruta del ejercicio**

**https://github.com/raco-dot/Actividad5-Banca-MLflow/blob/main/CHANGELOG.md**

| | |
|---|---|
| **Materia:** | *Gestión de Proyectos de Inteligencia Artificial* |
| **Actividad:** | Realizar la preparación de los datos, el entrenamiento de un modelo y el registro de los resultados. |
| **Alumno(a):** | *ROGELIO ACO GONZALEZ* |
| **Matrícula:** | *2987969* |
| **Docente:** | *LUIS ARIEL VAZQUEZ PIÑA* |
| **Fecha de entrega:** | *10/ NOV /2026* |

# Declaración de Uso de Inteligencia Artificial

El alumno debe indicar de manera honesta si utilizó herramientas de Inteligencia Artificial (IA) para el desarrollo de esta actividad. Completa la siguiente tabla:

| Herramienta de IA utilizada | ¿La usaste? | Propósito / Descripción de uso |
|---|---|---|
| ChatGPT / OpenAI | ■ Sí ☐ No | ● Elaboración del programa<br>● Me fue guiando paso a paso para aprender a utilizar el GitHub y me dio instrucciones para su llenado<br>● Me ayudó para extraer los datos de MLflow porque la pantalla no abrió de forma directa |
| Claude (Anthropic) | ☐ Sí ☐ No | |
| Gemini (Google) | ☐ Sí ☐ No | |
| Copilot (Microsoft) | ☐ Sí ☐ No | |
| Otra (especifica): | ☐ Sí ☐ No | |

# Antecedentes

Para realizar esta actividad he decidido continuar trabajando con el entorno bancario y seleccioné el caso de predicción de contratación de depósitos a plazo

Los datos para el entrenamiento y el ejercicio se han tomado de https://archive.ics.uci.edu/dataset/222/bank+marketing y corresponden a un estudio sobre respuestas a campañas telefónicas de una entidad bancaria de otro país. La variable objetivo indica si el cliente contrató o no, un depósito a plazo.

Dado que se trata de contratar o no contratar y esto es un resultado binario, se puede resolver comparando el desempeño de un algoritmo de regresión logística vs uno de Random Forest.

# Objetivo

El objetivo es estimar la probabilidad de que un cliente contrate un depósito a plazo a partir de información disponible para campañas comerciales.
Se realizará la preparación de los datos, el entrenamiento de un modelo y el registro de los resultados.

# Resumen ejecutivo

Se compararon regresión logística y Random Forest para predecir la contratación de depósitos a plazo a partir de información disponible para campañas comerciales.

- Se emplearon 45.195 registros tras limpieza.
- En la **regresión logística** se obtuvo: **F1 = 0,3790 y recall = 0,6314**
- En Random Forest se obtuvo: F1 = 0,3066 y recall = 0,2004.
- **Random Forest tuvo mayor accuracy (0,8939) y ROC-AUC (0,8033)**. La elección final dependerá del costo de la contactabilidad y del objetivo comercial.

# Diccionario de Datos

| Variable(s) | Significado |
|---|---|
| age, balance | Edad y saldo |
| job, marital, education | Ocupación, estado civil y educación |
| default, housing, loan | Indicadores de crédito impagado, hipoteca y préstamo personal |
| contact, day, month | Canal y calendario de contacto |
| campaign, pdays, previous | Historial y frecuencia de contactos |
| outcome | Resultado de campaña previa |
| duration | Duración de la llamada; EXCLUIDA por disponibilidad temporal |
| y | Objetivo: contratación de depósito (0/1) |

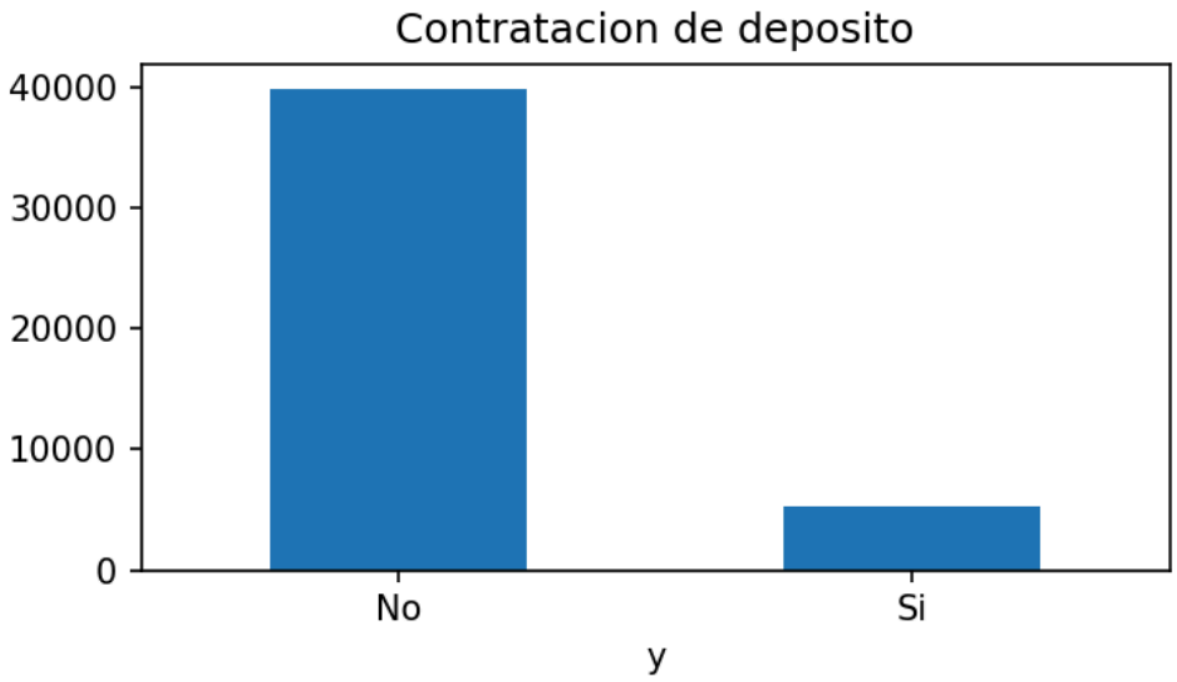
# Preparación limpieza y estandarización

El archivo original contiene 45,211 filas y 17 columnas. El script datos_prep.py comprueba la estructura esperada, se descartaron 16 duplicados, se normalizaron espacios y minúsculas en variables de texto y se transformó “y” a 0/1. El resultado fue 45,195 filas y 16 columnas (15 que son elementos para decidir y una de etiqueta).

Se ajustan sólo con datos de entrenamiento en cada partición o segmento. No se realiza eliminación automática de valores extremos; sin valores nulos de tipo NaN en el CSV procesado y las categorías textuales como unknown se conservan, sin tratarlas automáticamente como nulos. Esto lo realiza en el módulo train.py

Distribución del objetivo tras limpieza: 39,906 no contrataciones y 5,289 contrataciones; prevalencia positiva 11.70% (la prevalencia es el porcentaje de casos que presentan una determinada característica dentro de un conjunto de datos). Debido a este desequilibrio de datos, se recomienda interpretar F1, recall, precision y ROC-AUC junto a accuracy

Esta es la misma información mencionada, pero vista de forma gráfica. Los que no contratan después de la llamada y los que sí contratan.



# Arquitectura y configuración del programa

El proyecto está desarrollado en Python, utilizando Google Colab como entorno de ejecución, GitHub para el control de versiones y MLflow para el registro de experimentos.

El programa está organizado en tres componentes principales:

1. Programa principal (entrena)
2. Preparación de datos (datos_prep)
3. Entrenamiento y evaluación (train)

### 1. Programa principal (entrena.ipynb)

Coordina la ejecución del proyecto. Instala las librerías necesarias, configura MLflow, carga los archivos y ejecuta el script de entrenamiento train.py. Luego de esto, nos deja ver los los resultados y generar un ZIP con las evidencias.

### 2. Preparación de datos (datos_prep.py)

Es un módulo que está encargado de obtener y preparar el dataset Bank Marketing de UCI. Realiza la validación y limpieza de los registros, elimina duplicados exactos y normaliza variables.

### 3. Entrenamiento y evaluación (train.py)

Este programa es el que realmente tiene toda la lógica cob la que se dispara el aprendizaje automático. Sus principales etapas son:

- **Preprocesamiento:** identifica variables numéricas y categóricas, aplica imputación, estandarización y codificación mediante pipelines.
- **División de datos:** aplica la regla del 80/20, segmentando el 80 % de los datos para entrenamiento y el 20 % para prueba.
- **Definición de modelos:** configura regresión logística y Random Forest, junto con sus respectivos espacios de hiperparámetros.
- **Optimización:** utiliza GridSearchCV con validación cruzada estratificada de cinco folds y F1-score como criterio de selección. Evalúa 6 combinaciones de regresión logística y 8 de Random Forest.
- **Entrenamiento final:** selecciona automáticamente la mejor configuración y vuelve a entrenar el modelo con todos los datos de entrenamiento.
- **Evaluación:** calcula accuracy, precision, recall, F1-score y ROC-AUC sobre el conjunto de prueba reservado. También obtiene estadísticas F1 mediante validación cruzada.
- **Registro:** almacena en MLflow los hiperparámetros seleccionados, las métricas, las gráficas y los modelos entrenados.

**Como ultima etapa, se organizan los resultados**

Los archivos generados se distribuyen en carpetas específicas:

- datos/: conjuntos originales y procesados.
- resultados/: métricas comparativas y resultados de validación cruzada.
- graficas/: matrices de confusión, curvas ROC y visualizaciones exploratorias.
- evidencias_mlflow/: evidencias de los experimentos recuperadas mediante la API de MLflow.

**Al final, lo que logra el programa en su conjunto es: automatizar la comparación de los dos algoritmos de clasificación, la búsqueda de sus mejores hiperparámetros y el registro de resultados, incorporando prácticas básicas de MLOps.**

# Estrategia de entrenamiento y ajuste:

- División estratificada: 80 % entrenamiento y 20 % prueba, semilla aleatoria 42.
- Sobre entrenamiento se aplica StratifiedKFold de 5 particiones, con mezcla y semilla 42.
- GridSearchCV busca maximizar F1 y reentrena la mejor configuración.
- La prueba reservada se utiliza para evaluación final, no para elegir hiperparámetros.

Estos son lo hiperparámetros que controlan coómo aprende el modelo

| Modelo | Espacio de búsqueda | Mejor configuración |
|---|---|---|
| Regresión logística | Se probó con los siguientes valores<br>C = 0.1<br>C = 1<br>C = 10<br><br>class_weight: None / balanced | C = 10<br>class_weight = balanced |
| Random Forest | Árboles:<br>100<br>200<br>Profundidad:<br>8<br>16<br>min_samples_leaf:<br>1<br>5 | 200 árboles<br>Profundidad 16<br>hoja mínima 1 |

Se emplearon scikit-learn, Pandas, Matplotlib y MLflow; se registraron parámetros, métricas, matrices de confusión, curvas ROC y modelos serializados.

# Resultados y pruebas de validación cruzada

| Métrica | Regresión logística | Random Forest |
|---|---|---|
| Accuracy | 0.7578 | 0.8939 |
| Precision | 0.2708 | 0.6523 |
| Recall | 0.6314 | 0.2004 |
| F1-score | 0.379 | 0.3066 |
| ROC-AUC | 0.7735 | 0.8033 |
| F1 medio CV | 0.3713 | 0.3147 |

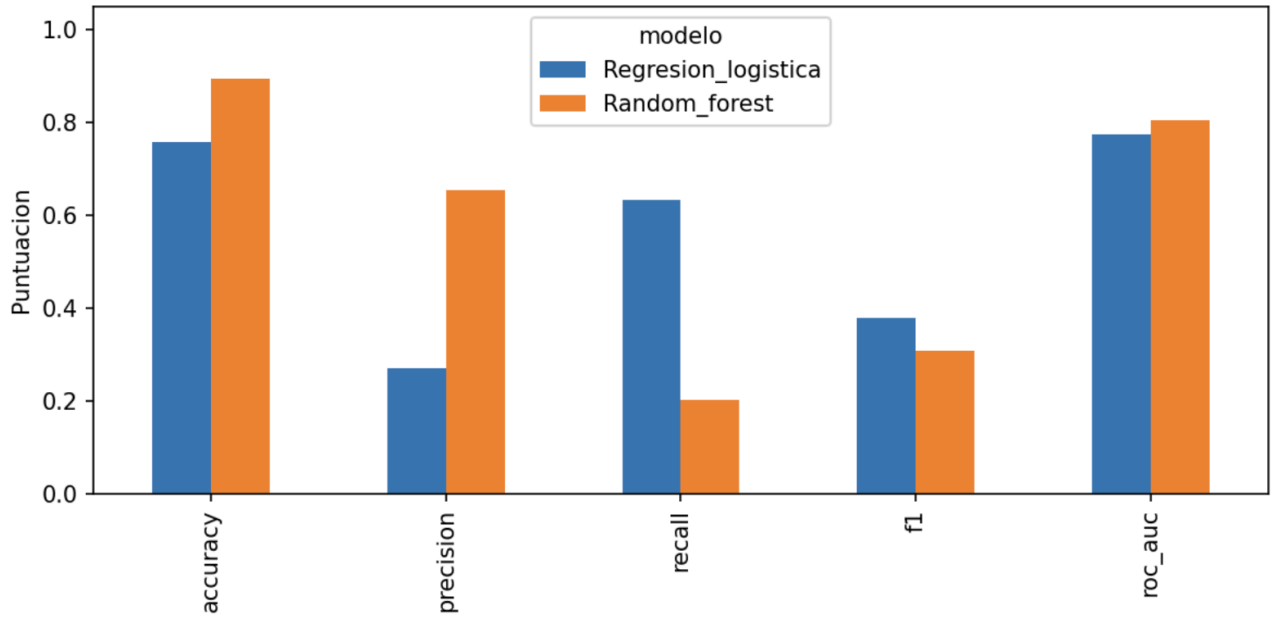
La regresión logística supera al Random Forest en F1 por 0.0724 puntos absolutos y en recall por 0.4310

Random Forest supera a la regresión logística en accuracy por 0.1361 y en ROC-AUC por 0.0298.

La clase positiva es minoritaria, por lo que accuracy por sí sola es insuficiente.

Gráficamente puede apreciarse mejor

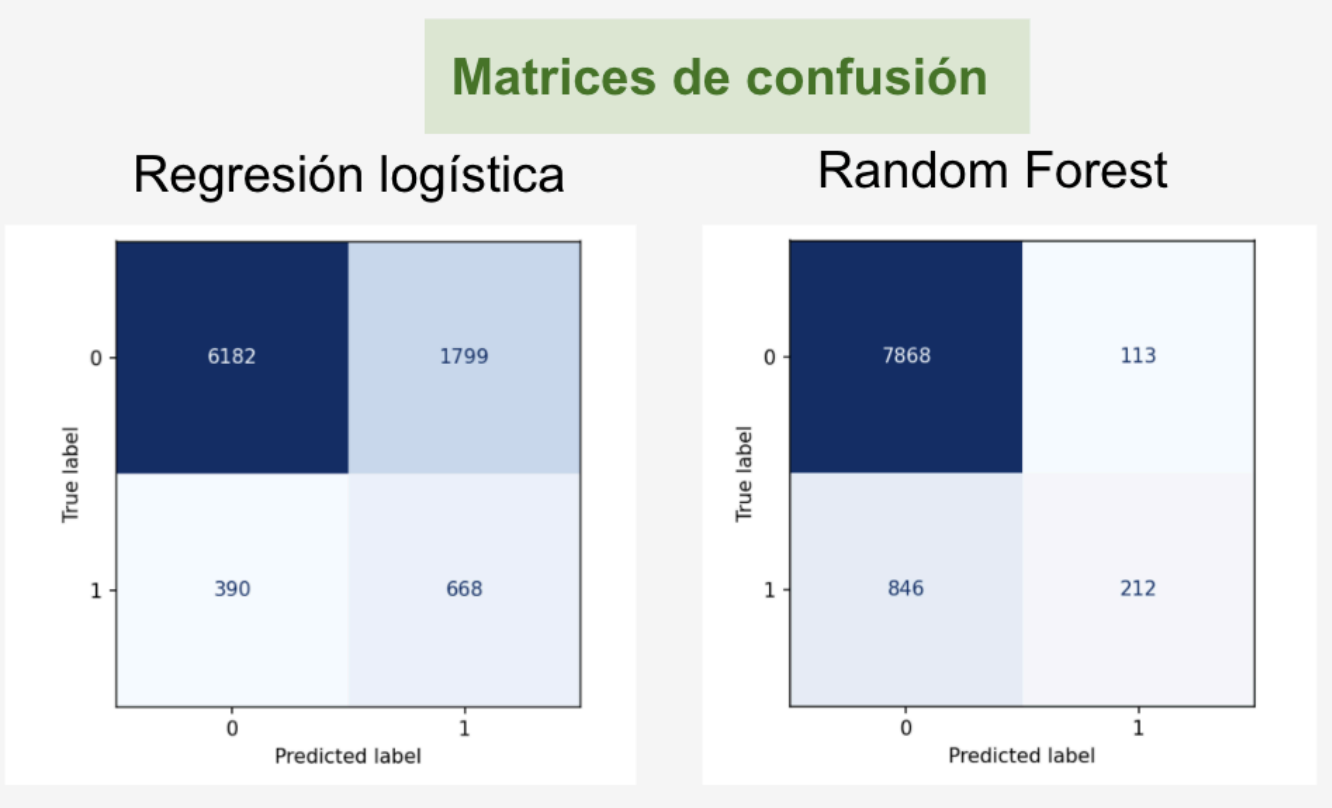
| | modelo | accuracy | precision | recall | f1 | roc_auc | cv_f1_mean | cv_f1_std | best_params |
|---|---|---|---|---|---|---|---|---|---|
| 0 | Regresion_logistica | 0.757827 | 0.270774 | 0.631380 | 0.379007 | 0.773500 | 0.371326 | 0.008498 | {"model__C": 10.0, "model__class_weight": "bal... |
| 1 | Random_forest | 0.893904 | 0.652308 | 0.200378 | 0.306580 | 0.803305 | 0.314719 | 0.019949 | {"model__max_depth": 16, "model__min_samples_l... |




En nuestro proyecto de predicción de contratación de depósitos bancarios, la **regresión logística** sería la mejor opción si buscamos identificar al mayor número posible de clientes que realmente contratarían un depósito. Para aterrizarlo un poco más: de cada 100 clientes que realmente contratarían un depósito, regresión logística identifica aproximadamente 63. Esto es muy útil cuando queremos encontrar más oportunidades comerciales, aunque también genera más falsos positivos.

Sin embargo, **Random Forest** es mejor si buscáramos mayor precisión en los clientes seleccionados. Igual, aterrizando un poco mas: de cada 100 clientes que realmente contratarían, Random Forest identifica aproximadamente 20. Lo relevante de este modelo es que entre los clientes que predice como posibles contratantes, tiene una precisión considerablemente mayor.

Como referencia incluiré las dos matrices de confusión, en donde se observa que la regresión logística es **mejor para identificar un mayor porcentaje de los clientes que realmente contratarían un depósito** (aunque como ya vimos no es mejor en todo)



# Elección final

Me inclino por regresión logística por las siguientes razones:

1. Obtiene mayor F1-score, que fue precisamente la métrica utilizada para seleccionar los hiperparámetros mediante GridSearchCV.
2. Tiene mayor recall, por lo que identifica más contrataciones reales.
3. Es un modelo generalmente más interpretable, algo valioso en aplicaciones bancarias.
4. De acuerdo a referencias reales y actuales al momento de este ejercicio, el costo de contactar clientes no es muy elevado, el monto mayor lo gasta el banco al ofrecer la comisión por venta o contratación realizada.

# Evidencias de MLFlow

Se utilizó MLflow, pero no fue posible visualizarlo en pantalla desde Google Colab, sin embargo sí se utilizó para guardar los resultados de las diferentes ejecuciones y por medio de la API se pudieron recuperar desde Google Colab los resultados almacenados. En el siguiente cuadro se muestran los resultados de ambos modelos y se tomará la ejecución exitosa en cada caso como referencia.

```
display(ejecuciones[columnas])
```

```
EXPERIMENTO: Actividad5_Banca
TOTAL DE EJECUCIONES: 5
```

| | tags.mlflow.runName | status | metrics.accuracy | metrics.precision | metrics.recall | metrics.f1 | metrics.roc_auc |
|---|---|---|---|---|---|---|---|
| 0 | Random_forest | FINISHED | 0.893904 | 0.652308 | 0.200378 | 0.306580 | 0.803305 |
| 1 | Regresion_logistica | FINISHED | 0.757827 | 0.270774 | 0.631380 | 0.379007 | 0.773500 |
| 2 | Regresion_logistica | FAILED | 0.757827 | 0.270774 | 0.631380 | 0.379007 | 0.773500 |
| 3 | Regresion_logistica | FAILED | 0.757827 | 0.270774 | 0.631380 | 0.379007 | 0.773500 |
| 4 | Regresion_logistica | FAILED | 0.757827 | 0.270774 | 0.631380 | 0.379007 | 0.773500 |

Durante el desarrollo se registraron cinco ejecuciones en MLflow: dos finalizadas correctamente y tres con estado FAILED. Las tres ejecuciones fallidas de regresión logística conservaron los mismos valores de métricas que la ejecución finalizada correctamente de ese algoritmo, identificada con el índice 1 en la tabla. Aunque sus métricas fueron calculadas y registradas, se presentaron dificultades técnicas durante la finalización del proceso, posiblemente relacionadas con la serialización del modelo.

Posteriormente se ajustó el código para utilizar cloudpickle, logrando completar correctamente el registro de ambos algoritmos. Se conservaron las ejecuciones fallidas como evidencia de trazabilidad (posiblemente relacionadas con la serialización), y de la evolución técnica del proyecto y vale la pena apreciar que sus resultados al momento de que fallaron fue los mismos que los de la ejecución 2 que

resultó exitosa. Esta información permite documentar las condiciones bajo las cuales se entrenaron ambos modelos y contribuye a su reproducibilidad.

En el siguiente cuadro se muestra la configuración final con la que se entrenaron los modelos, luego de que GridSearchCV seleccionara los mejores hiperparámetros.

HIPERPARÁMETROS REGISTRADOS EN MLFLOW

| | 0 | 1 |
|---|---|---|
| **tags.mlflow.runName** | Random_forest | Regresion_logistica |
| **params.cv_folds** | 5 | 5 |
| **params.dataset** | UCI bank-full.csv sin duration | UCI bank-full.csv sin duration |
| **params.max_depth** | 16 | None |
| **params.n_estimators** | 200 | None |
| **params.random_state** | 42 | 42 |
| **params.test_size** | 0.2 | 0.2 |
| **params.min_samples_leaf** | 1 | None |
| **params.C** | None | 10.0 |
| **params.class_weight** | None | balanced |

Este último cuadro muestra los archivos generados durante la evaluación de cada modelo y recuperados del respaldo de MLflow. Demuestra que pudimos recuperar los archivos de evaluación asociados a las ejecuciones exitosas, que permiten analizar y documentar el desempeño de los modelos.

```
...
    Modelo: Random_forest
    Artifact URI registrada: /content/mlruns/1/4ecea60639cd4163ab777e6b74a355db/artifacts
    Carpeta restaurada: /content/mlflow_recuperado/Actividad5/evidencias/mlruns/1/4ecea60639cd4163ab777e6b74a355db/artifacts
    Existe: True
     - graficas/roc_Random_forest.png
     - graficas/matriz_Random_forest.png

    Modelo: Regresion_logistica
    Artifact URI registrada: /content/mlruns/1/0bb2909ac4c8403384f59e616a468685/artifacts
    Carpeta restaurada: /content/mlflow_recuperado/Actividad5/evidencias/mlruns/1/0bb2909ac4c8403384f59e616a468685/artifacts
    Existe: True
     - graficas/roc_Regresion_logistica.png
     - graficas/matriz_Regresion_logistica.png
```

## Reflexión sobre deuda técnica

Entendiendo la deuda técnica como mejoras técnicas que es conveniente implementar o manualidades que es conveniente evitar o automatizar,  este ejericio ha resultado muy productivo, ya que nos permitió preparar un conjunto de datos bancarios, entrenar y comparar dos algoritmos de clasificación mediante búsqueda de hiperparámetros y validación cruzada, así como registrar parámetros, métricas y artefactos con MLflow.

Los resultados nos indican que es mejor la la regresión logística cuando se busca maximizar el F1-score y la identificación de contrataciones reales, pero por otro lado Random Forest obtuvo mejores resultados en accuracy, precision y ROC-AUC. Lo anterior no demuestra una superioridad estadística concluyente ni el beneficio económico de aplicar alguno de los modelos en una campaña comercial real (aunque antes yo me haya inclinado por la regresión logística).

Desde la perspectiva de MLOps, el proyecto ha permitido identificar algunas limitaciones técnicas. Entre ellas destacan la dependencia del almacenamiento temporal de Google Colab, la necesidad de recuperar los registros de MLflow desde archivos de respaldo, las dificultades de acceso a su interfaz gráfica y la dependencia de rutas específicas para ejecutar el programa.

Estas limitaciones representan oportunidades de mejora que podrían resolverse mediante almacenamiento persistente, automatización de procesos, pruebas técnicas y una infraestructura de seguimiento de experimentos más estable.

El proyecto se publicó en GitHub junto con sus datos, el código, las métricas, las visualizaciones y las evidencias recuperadas mediante la API de MLflow. Aunque no fue posible obtener capturas de su interfaz gráfica debido a problemas de acceso mediante el proxy de Colab, limitación que fue mencionada previamente en este documento.

En conclusión, el proyecto cumplió sus objetivos principales de preparación de datos, entrenamiento, evaluación y registro de resultados, a la vez que permitió comprender la importancia de la reproducibilidad, la trazabilidad y la gestión de deuda técnica en proyectos de inteligencia artificial.